

A 图论选择练习题

A 图论选择题练习

时间限制：1000ms 内存限制：65536kb

通过率：200/201 (99.50%) 正确率：200/202 (99.01%)

选择题作答要求

请根据题目要求完成下列选择题，并将答案以每行一个只包含大写英文字母的、字母按照升序的字符串的格式输出，不要输出多余空格。假设有 n 道题目，那么 Special Judge 只会读取你输出的前 n 行内容，如果输出不足 n 行也会被判定为格式错误。

例如，有三道选择题，第一题答案为 BCE，第二题答案为 AC，第三题答案为 D，你应该输出以下答案。

```
BCE
AC
D
```

你可以使用如下的代码进行输出：

```
#include <stdio.h>

int main(){
    puts("BCE");
    puts("AC");
    puts("D");
}
```

只要满足上述格式要求，即返回 Accepted 结果，不返回真实评测结果，待比赛结束后正式评测使用每个人的最后一次 Accepted 的提交记录进行正式评测。

如果格式错误，会返回 Presentation Error 结果。

题目描述

第一题

下面关于 Dijkstra 算法不恰当的描述有_____。

- A. 采用 DP 方法。
- B. 采用分治方法。
- C. 采用贪心策略。
- D. 利用了边的松弛操作。
- E. 利用了优先队列。
- F. 顶点遍历是采用 DFS。

第二题

基于拓扑排序的最短路径算法适用于哪种性质的图_____。

- A. 有向、无回路、有负权边。
- B. 有向、无回路、无负权边。
- C. 有向、有回路、有负权边。
- D. 有向、有回路、无负权边。

第三题

Bellman-Ford、基于拓扑排序、采用 std::priority_queue 的 Dijkstra 三种算法的时间复杂度分别为（采用邻接表存储图，设点数和边数分别为 n 和 m ）_____。

- A. $O(nm)$ 、 $O(n^2)$ 、 $O(n+m)$ 。
- B. $O(n^2)$ 、 $O(nm)$ 、 $O(n+m)$ 。
- C. $O(nm)$ 、 $O(n+m)$ 、 $O(n+m)$ 。
- D. $O(nm)$ 、 $O(n+m)$ 、 $O(n\log n+m)$ 。
- E. $O(n^2)$ 、 $O(n+m)$ 、 $O(n\log n+m)$ 。
- F. $O(n^2)$ 、 $O(n+m)$ 、 $O(n^2)$ 。

B

B 运动会观赛安排

时间限制：1000ms 内存限制：65536kb

通过率：184/200 (92.00%) 正确率：184/726 (25.34%)

题目描述

钢条厂终于完成北航 70 周年校庆的大订单，因此也让员工休息几天。Zhoues 打算借这个空闲的时间回北航看看正在进行的运动会。

Zhoues 对所有的运动会比赛都很感兴趣，想为软件学院的运动健儿们加油鼓劲。Zhoues 一旦为某一场比赛加油鼓劲，他就必须全程看完这次比赛，但是很可惜的是有相当多的比赛项目的时间安排是重叠的，Zhoues 希望可以好好规划一下观赛的时间安排，让他在有限的时间内为尽可能多的比赛加油鼓劲，那么最多他可以为多少场比赛加油鼓劲呢？

形式化地说，共有 n 场比赛，第 i 场比赛的时间可以用左闭右开区间 $[x_i, y_i)$ 来表示，你需要选择尽可能多的比赛使得任意两个不同比赛对应的区间交集均为空。

输入格式

第一行一个正整数 n ($1 \leq n \leq 10^6$)，表示运动会的比赛数量。

接下来 n 行，每行两个整数 x_i, y_i ($1 \leq x_i < y_i \leq 10^6$)，表示第 i 场比赛的时间。

输出格式

一行一个正整数，表示 Zhoues 最多可以为运动会比赛加油鼓劲的次数。

样例输入

```
3
2 4
3 5
4 6
```

样例输出

```
2
```

样例解释

一种最优方案为选择第 1 场比赛和第 3 场比赛。

Hint

大胆贪心，小心求证！

经典贪心问题，本题的所有伪代码具体可以参考算法导论 P238 或者课件 PPT。

```
#include<bits/stdc++.h>
using namespace std;

pair<int,int>a[1000005];

int cmp(pair<int,int>a,pair<int,int>b){
    if(a.second!=b.second)return a.second<b.second;
    else return a.first<b.first;
}

int main(){
    using namespace std;
    int n;
    cin>>n;
    for(int i = 0;i < n;i++){
        cin>>a[i].first>>a[i].second;
    }
    sort(a,a+n,cmp);
    int ans = 1;
    int temp = a[0].second;
    for(int i=1;i<n;i++){
        if(a[i].first >= temp){
            temp = a[i].second;
            ans++;
        }
    }
```

```
    }
    cout<<ans;
}
```

C

C 偷钢条

时间限制：1000ms 内存限制：65536kb

通过率：144/194 (74.23%) 正确率：144/843 (17.08%)

题目描述

Zhoues 虽然是一名高级技师，但是其实他还暗中申请了安保工作，是一名光荣的安保大队队员。一天晚上，发现有一个人影在流水线附近，他悄悄的走进一看，发现是一个正在偷钢条的小偷！

小偷嘴里默念着：「我现在有一个最大承重为 k 吨的背包，现在流水线上有 n 根钢条，第 i 根钢条重量为 w_i 吨，价值为 v_i ，我最终的收益是偷到钢条的价值之和，我偷到钢条的总重量不能超过我背包的最大承重，那我最终可以获得的最大收益是多少呢？」

Zhoues 其实在瞬间就计算完毕了，但是他不确定是不是正确的，于是他希望你帮忙计算一下小偷最终获得的最大收益是多少。

输入格式

第一行两个正整数 n, k ($1 \leq n \leq 10^3, 1 \leq k \leq 10^5$)，表示钢条的数量和小偷背包的最大承重。

接下来 n 行，每行两个正整数 v_i, w_i ($1 \leq v_i \leq 10^7, 1 \leq w_i \leq 10^6$)，表示第 i 根钢条的价值和重量。

输出格式

一行一个非负整数，表示小偷可以获得的最大收益。

输入样例

```
3 6
1 1
1 3
2 4
```

输出样例

```
3
```

样例解释

一种最优方案为偷走第 1 根和第 3 根钢条。

Hint

注意是 n 根，不是 n 种。

经典动态规划问题，本题具体可以参考算法导论 P244 或者课件 PPT。

```
#include<iostream>
#include<cstring>
using namespace std;
long long dp[200001];
long long v[300001],w[300001];
long long i,j,m,n;
int main(){
    scanf("%lld%lld",&n,&m);

    dp[0]=0;
    for(i=1;i<=n;i++)
    {
        scanf("%lld %lld",&v[i],&w[i]);
    }
    for(i=1;i<=n;i++)
    {
```

```
        for(j=m;j>=w[i];j--){
            if(dp[j]<dp[j-w[i]]+v[i] )
                dp[j]=dp[j-w[i]]+v[i];
        }
    }
    printf("%lld\n",dp[m]);

    return 0;
}
```

D

D 偷沙砾

时间限制：1000ms 内存限制：65536kb

通过率：169/181 (93.37%) 正确率：169/331 (51.06%)

题目描述

Zhoues 虽然是一名高级技师，但是其实他还暗中申请了安保工作，是一名光荣的安保大队队员。一天晚上，发现有一个人在钢条厂原料区附近，他悄悄的走进一看，发现是一个正在偷沙砾的小偷！

小偷嘴里默念着：「我现在有一个最大承重为 k 吨的背包，现在原料区上有 n 种沙砾，第 i 种沙砾的总重量为 w_i 吨，总价值为 v_i ，如果偷走 j ($0 < j \leq w_i$) 吨第 i 种沙砾，那么我能获得的收益是 $\frac{jv_i}{w_i}$ ，我最终的收益是偷到沙砾的价值之和，我偷到沙砾的总重量不能超过我背包的最大承重，那我最终可以获得的最大收益是多少呢？」

Zhoues 其实在瞬间就计算完毕了，但是他不确定是不是正确的，于是他希望你帮忙计算一下小偷最终获得的最大收益是多少。

输入格式

第一行两个正整数 n, k ($1 \leq n \leq 10^3, 1 \leq k \leq 10^5$)，表示沙砾的种数和小偷背包的最大承重。

接下来 n 行，每行两个正整数 v_i, w_i ($1 \leq v_i \leq 10^7, 1 \leq w_i \leq 10^6$)，表示第 i 种沙砾的总价值和总重量。

输出格式

一行一个保留三位小数的实数，表示小偷可以获得的最大收益。

输入样例

```
3 6
1 1
1 3
2 4
```

输出样例

```
3.333
```

输出样例

```
3.333
```

样例解释

一种最优方案为偷走 1 吨第 1 种沙砾、1 吨第 2 种沙砾和 4 吨第 3 种沙砾。

Hint

大胆贪心，小心求证！

注意是 n 种，不是 n 根。

经典贪心问题，本题具体可以参考算法导论 P244 或者课件 PPT。

```
#include<cstdio>
#include<algorithm>
using namespace std;
int i,j,cnt = 0,n;
```

```
double w,v,k,res,add;
struct node {
    double w;
    double rate;
    inline bool operator < (const node& o)const {
        return o.rate < rate;
    }
} Node[10007];
int main() {

    scanf("%d %lf",&n,&k);

    for(int i = 0; i<n; i++) {
        scanf("%lf %lf",&v,&w);
        Node[i].w = w;
        Node[i].rate = v/w;
    }
    sort(Node,Node+n);
    cnt = 0;
    res = 0.0;

    while(k>0.0 && cnt < n && Node[cnt].rate>0) {
        add = min(k , Node[cnt].w);
        res += add*Node[cnt++].rate;
        k-=add;
    }

    printf("%.3lf\n",res);

    return 0;
}
```

E

E 暗中加密通话

时间限制：1000ms 内存限制：65536kb

通过率：29/50 (58.00%) 正确率：29/161 (18.01%)

题目描述

Zhoues 虽然是一名高级技师，但是依旧摆脱不了摸鱼的习惯。他听说小水獭已经进厂切钢条了，就私下和小水獭商量看看能不能设计一套加密方式加密二人间的通话，这样就可以方便摸鱼啦！

最后二人决定使用哈夫曼编码来给通话加密。数据结构这门课程中介绍到：「哈夫曼编码可以构造出字符的最优二进制表示」。那么给定一个长度为 n 的字符串，能不能求出该字符串对应的二进制哈夫曼编码的总长度呢？

输入格式

一行一个仅包含大小写英文字母的非空字符串，长度不超过 10^7 。

输出格式

一行一个正整数，表示该字符串对应的二进制哈夫曼编码的总长度。

输入样例 1

abababac

输出样例 1

12

输入样例 2

vOEwOVQQtIPatwEZPPOKtApNDdpJEFQChpJwNqQAFDYvwlFMNOpqKczuWOTBxCJNaVadetEVCaIhHJKIcBoEfsUqRUBGHZucFZlU

voEwOVQQtIPatwEZPPOKtApMDdpjEFQChpJwNqqAFDYvwLFWMOpqXczuWotBxCiNavadetEVCaihHJkI
cBoEFsUqRubGHZucfZlU

输出样例 2

535

样例解释

第一个样例中，a 出现了 4 次，b 出现了 3 次，c 出现了 1 次，加密后 a,b,c 的哈夫曼编码分别为 0,11,10 或 0,10,11，但总编码长度均为 $1 \times 4 + 2 \times 3 + 2 \times 1 = 12$ 。

Hint

大胆贪心，小心求证！

经典优先队列贪心问题，具体可以参考算法导论 P245 页。

```
#include<bits/stdc++.h>
#define maxn 10000002
using namespace std;
char s[maxn];
int fre[100],ans,t,i,sum,a,b,l;
priority_queue <int,vector<int>,greater<int> > q;
int main() {
    scanf("%s",s);
    l = strlen(s);
    for(i=0; i<l; i++) {
        fre[s[i]-'A']++;
    }
    for(i=0; i<100; i++) {
        if(fre[i]!=0) {
            q.push(fre[i]);
            sum++;
        }
    }
    if(sum==1) {
        printf("%d",l);
        return 0;
    }
    while(q.size()>1) {
        a = q.top();q.pop();
        b = q.top();q.pop();
        ans+=(a+b);q.push(a+b);
    }
    printf("%d",ans);
}
```

F 上班 I

时间限制：2000ms 内存限制：65536kb

通过率：79/114 (69.30%) 正确率：79/329 (24.01%)

题目描述

观赛完运动会之后，Zhoues 需要立马回去钢条厂上班，由于北航到钢条厂的路径有很多条，他希望选择一条最短的！

形式化来说，给你一个 n 个点 m 条边的带权无向图，求点 s 到点 t 的最短路长度。

输入格式

第一行四个正整数 n, m, s, t ($1 \leq n \leq 2.5 \times 10^3, 1 \leq m \leq 7 \times 10^3, 1 \leq s, t \leq n$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^3$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

图中有可能存在重边和自环。

输出格式

输出一行一个非负整数，表示点 s 到点 t 的最短路长度。

输入样例

```
7 11 5 4
2 4 2
1 4 3
7 2 2
3 4 3
5 7 5
7 3 3
6 1 1
6 3 4
2 4 3
5 6 3
7 2 1
```

```
7 11 5 4
2 4 2
1 4 3
7 2 2
3 4 3
5 7 5
7 3 3
6 1 1
6 3 4
2 4 3
5 6 3
7 2 1
```

输出样例

```
7
```

Hint

请注意重边和自环的处理，不需要对最短路算法进行优化就可以通过本题。

经典图论问题，该题所有伪代码可以见算法导论或 PPT。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll read() {
    ll x=0,w=1; char c=getchar();
    while (c<'0' || c>'9') { if (c=='-') w=-1; c=getchar(); }
    while (c>='0' && c<='9') x=x*10+c-'0',c=getchar();
```

```

        return X*w;
    }

    const ll N=10000000+10,M=15000000+10;

    ll n,m,s,t;
    struct edge { ll v,w,nxt; } e[M];
    ll head[N];
    void addEdge(ll u,ll v,ll w) {
        static ll cnt=0;
        e[++cnt]=(edge){v,w,head[u]},head[u]=cnt;
    }

    priority_queue<pair<ll,ll>,vector<pair<ll,ll>>,greater<pair<ll,ll>>> Q;
    ll dis[N];
    void dijkstra(ll s) {
        memset(dis,0x3f,sizeof(dis));
        dis[s]=0,Q.push(make_pair(0,s));
        while (!Q.empty()) {
            ll u=Q.top().second,d=Q.top().first; Q.pop();
            if (d!=dis[u]) continue;
            for (ll i=head[u];i;i=e[i].nxt) {
                ll v=e[i].v,w=e[i].w;
                if (dis[u]+w<dis[v]) dis[v]=dis[u]+w,Q.push(make_pair(dis[v],v));
            }
        }
    }

    int main() {
        n=read(),m=read(),s=read(),t=read();
        for (ll i=1;i<=m;++i) {
            ll u=read(),v=read(),w=read();
            addEdge(u,v,w);
            addEdge(v,u,w);
        }
        dijkstra(s);
        printf("%lld",dis[t]);
        return 0;
    }

```

题目描述

小水獭正在浏览沙河校区的蓝园（Blue Graden）。

小水獭想将蓝园中 n 朵花发往学院路，这 n 朵花的重量分别为 $a_1, a_2, \dots, a_n$ ，一个快递可以运送任意一朵花，或运送两朵重量之和不超过 m 的花。

小水獭想知道，它最少需要发几个快递才能将所有的花发往学院路，你能帮帮它嘛？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 10^9$)，含义同题目描述。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq m$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个正整数，表示最少需要的快递数量。

输入

2
5 5
1 2 3 4 5
2 3
2 2

输出

3
2

样例解释

第一组数据中，一种最优方案为 $[1, 4], [2, 3], [5]$ 。

第二组数据中，一种最优方案为 $[1], [2]$ 。

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;
int a[maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) scanf("%d", &a[i]);
        sort(a + 1, a + 1 + n);
        int ans = 0, l = 1, r = n;
        while(l <= r){
            if(l == r || a[l] + a[r] <= m) l++, r--;
            else r--;
        }
        ans++;
    }
    printf("%d\n", ans);
}
```

```
        ans++;
    }
    printf("%d\n", ans);
}
}
```

H

H XIAO7 和奇怪的伞

时间限制：1000ms 内存限制：65536kb

通过率：9/17 (52.94%) 正确率：9/53 (16.98%)

题目描述

10^9 只猫猫站成了一排，其中前 m 只是属于 XIAO7 的。这时突然下雨了，XIAO7 变出了 n 张奇怪的大伞，其中第 i 张大伞能遮住第 $a_i, a_i + 1, \dots, b_i - 1, b_i$ 只猫猫，请问最少用多少张大伞才能遮住所有 XIAO7 的猫猫。

如果无论如何都不能遮住所有 XIAO7 的猫猫，请输出 $(=> <=)$ 。

输入格式

第一行两个整数 n, m ($1 \leq n \leq 10^6, 0 \leq m \leq 10^9$)，含义同题目描述。

接下来 n 行，每行两个正整数 a_i, b_i ($1 \leq a_i \leq b_i \leq 10^9$)，表示第 i 张大伞能遮住的猫猫范围。

输出格式

输出一行：

- 若存在一种方案可以遮住所有 XIAO7 的猫猫，输出一个非负整数，表示最少需要的伞的数量。
- 否则，输出 $(=> <=)$ 。

输入样例

```
3 5
1 2
2 4
3 6
```

输出样例

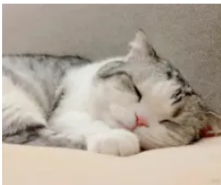
```
2
```

样例解释

一种最优方案为选择第 1 张伞和第 3 张伞。

Hint

XIAO7：发出喵喵的声音~



AUTHOR: YUKI

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
```

```

#include<set>
#include<ctime>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 1000005;
const int inf = 0x3f3f3f3f;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
pi a[N];
int main(){
    int n = getint(), t = getint();
    for(int i = 1; i <= n; i ++){
        a[i].first = getint(), a[i].second = getint();
        sort(a + 1, a + n + 1);

        if(!t) {
            puts("0"); return 0;
        }

        int ed = 0, ans = 0;
        for(int i = 1; i <= n && ed < t;) {
            int lasted = ed + 1;
            for(; a[i].first <= lasted && i <= n; i ++){
                ed = max(ed, a[i].second);
            }
            if(lasted == ed + 1) break ;
            ans ++;
        }
        if(ed < t)
            puts("(=> <=)");
        else cout << ans << endl;
    }
}

```

I 莫卡的蛇蛇数列

时间限制: 1000ms 内存限制: 65536kb

通过率: 5/7 (71.43%) 正确率: 5/10 (50.00%)

题目描述

Serval 是加帕里幼儿园的新生，而莫卡是旭丘幼儿园的小朋友。

莫卡有一个长度为 n 的数列 $a_1, a_2, \dots, a_n$ ，Serval 想将这个数列变为**蛇蛇数列**。对于一个数列 $a_1, a_2, \dots, a_n$ ，Serval 定义这个数列是蛇蛇数列当且仅当这个数列满足以下条件：

- 对于满足 $2k \leq n$ 的所有正整数 k ，有 $a_{2k} > a_{2k-1}$ 。
- 对于满足 $2k + 1 \leq n$ 的所有正整数 k ，有 $a_{2k} > a_{2k+1}$ 。

为了将莫卡的数列变为蛇蛇数列，Serval 每次可以选择某个 a_i ，花费 1 的代价将 a_i 变为 $a_i + 1$ 或者 $a_i - 1$ 。需要注意的是，蛇蛇数列中的元素不必一定为正整数。

聪明的 Serval 当然知道将莫卡的数列变为蛇蛇数列的最小代价，但是莫卡并不知道。你能帮莫卡求出最小代价吗？

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^5$)，表示莫卡的数列长度。

第二行 n 个正整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$)，表示莫卡的数列元素。

输出格式

对于每组数据，输出一行一个非负整数，表示将莫卡的数列变为蛇蛇数列的最小代价。

输入样例

```
2
3
1 2 3
4
```

输出样例

```
2
6
```

样例解释

第一组数据中，可以选择 a_3 ，对其进行 -1 操作 2 次，最后的序列为 $[1, 2, 1]$ 。

第二组数据中，可以先选择 a_2 ，对其进行 $+1$ 操作 4 次，再选择 a_4 ，对其进行 $+1$ 操作 2 次，最后的序列为 $[4, 5, 3, 4]$ 。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int t;
    scanf("%d", &t);
    while(t--){
        int n,i,x;
        long long Ans=0;
        scanf("%d",&n);
        scanf("%d",&x);
        for (i=2;i<=n;i++){
            int y;
            scanf("%d",&y);
            if (i&1){
                int t=max(y-x+1,0);
                y-=t;
                Ans+=t;
            }else{
                int t=max(x+1-y,0);
                y+=t;
                Ans+=t;
            }
        }
    }
}
```

```
        x=y;
    }
    printf("%lld\n",Ans);
}
return 0;
}
```

J 小水獭和最大值 III

时间限制：1000ms 内存限制：65536kb

通过率：2/4 (50.00%) 正确率：2/6 (33.33%)

题目描述



小水獭正在学习动态规划。

给定一个 $n \times m$ 的矩阵 $\{a_{i,j}\}_{n \times m}$ ，小水獭初始位于 $(1,1)$ ，目标位置为 (n,m) ，它每次可以向下移动或向右移动，即从 (x,y) 移动到 $(x+1,y)$ 或 $(x,y+1)$ ，但不能移动到矩阵之外。它一共经过了 $n+m-1$ 个位置，将这些位置的元素进行按位与运算得到 A ，求 A 的最大值。

输入格式

第一行一个正整数 t ($1 \leq t \leq 100$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n, m \leq 200$)，含义同题目描述。

接下来 n 行，第 i ($1 \leq i \leq n$) 行 m 个整数 $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($0 \leq a_{i,j} < 2^{30}$)，含义同题目描述。

输出格式

对于每组数据，输出一行一个非负整数，表示 A 的最大值。

输出格式

对于每组数据，输出一行一个非负整数，表示 A 的最大值。

输入样例

```
2
2 2
1 2
3 4
2 2
7 4
3 1
```

输出样例

```
0
1
```

```
#include <stdio.h>
#include <string.h>
#define maxn 205

int n, m;
int a[maxn][maxn];
int tag[maxn][maxn], g[maxn][maxn];
int f[maxn][maxn];

int solve(){
```

```

    if(tag[1][1]) return 0;
    f[1][1] = 1;
    int i, j;
    for(i = 1; i <= n; i++){
        for(j = 1; j <= m; j++){
            if(tag[i][j]) continue;
            f[i][j] |= f[i - 1][j] | f[i][j - 1];
        }
    }
    return f[n][m];
}

int main(){
    int t, ans, i, j, x;
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        memset(tag, 0, sizeof(tag));
        ans = 0;
        for(i = 1; i <= n; i++){
            for(j = 1; j <= m; j++){
                scanf("%d", &a[i][j]);
            }
        }
        for(x = 1 << 29; x >= 1){
            memset(f, 0, sizeof(f));
            memset(g, 0, sizeof(g));
            for(i = 1; i <= n; i++){
                for(j = 1; j <= m; j++){
                    if(tag[i][j]) continue;
                    if(!(a[i][j] & x)){
                        g[i][j] = tag[i][j] = 1;
                    }
                }
            }
            if(solve()){
                ans += x;
            }else{
                for(i = 1; i <= n; i++){
                    for(j = 1; j <= m; j++){
                        if(g[i][j]) tag[i][j] = 0;
                    }
                }
            }
        }
        printf("%d\n", ans);
    }
}

```

K XIAO7 和奇怪的猫窝

时间限制：1000ms 内存限制：65536kb

通过率：5/6 (83.33%) 正确率：5/20 (25.00%)

题目描述

XIAO7 购买了一个奇怪的猫窝，虽然它能够装下全部的猫猫，但是太拥挤了还是会影响猫猫的心情。

第 i 只猫猫单独住进猫窝的快乐程度是 a_i ，除它本身每多住进来一只猫猫，她的快乐程度就会减去 b_i 。

现在 XIAO7 想知道选择哪些猫猫，能使猫窝里所有猫的快乐程度之和最大。

输入格式

第一行一个整数 n ($1 \leq n \leq 10^3$)，表示猫猫的数量。

接下来 n 行，每行两个整数 a_i, b_i ($1 \leq a_i, b_i \leq 10^9$)，含义同题目描述。

输出格式

第一行一个正整数表示最大的快乐程度之和。

第二行一个正整数表示选中的猫猫数量，如果有多种可能的答案，请输出猫猫数量的最大值。

输入样例

```
5
10 2
10 2
10 2
10 2
9 3
```

输出样例

```
18
3
```

样例解释

选择 1, 2, 3 号猫猫，每只猫猫的快乐程度都是 $10 - 2 \times 2 = 6$ ，快乐程度之和为 18。

Hint

XIAO7：发出喵喵的声音~



AUTHOR: YUKI

```
#include<bits/stdc++.h>
using namespace std;
const int N = 2000;
char s[N];
long long a[N];
struct fgo{
    int x;
    int y;
}f[N];
bool cmp(long long x,long long y)
{
    return x>y;
}
int main()
{
```

```

int t=1,n;
while(t--)
{
    scanf("%d",&n);
    for(int i=0;i<n;i++)
    {
        scanf("%d%d",&f[i].x,&f[i].y);
    }
    long long ans=0;
    int ansi=1;
    for(int i=n;i>=1;i--)
    {
        long long tmp=0;
        for(int j=0;j<n;j++)
        {
            a[j]=(long long)f[j].x-(long long)f[j].y*(i-1);
        }
        sort(a,a+n,cmp);
        for(int j=0;j<i;j++)
        {
            tmp+=a[j];
        }
        if(ans<=tmp)
        {
            ans=tmp;
            ansi=i;
        }
    }
    printf("%lld\n",ans);
    printf("%d",ansi);
}
}

```

L

L XIAO7 和奇怪的排列

时间限制: 1000ms 内存限制: 204800kb

通过率: 1/1 (100.00%) 正确率: 1/2 (50.00%)

题目描述

对于 1 到 n 的排列 $a_1, a_2, \dots, a_n$ 和 $b_1, b_2, \dots, b_n$, 定义 $\text{MAX}(a, b) = \sum_{i=1}^n \max(a_i, b_i)$ 。对于给定的非负整数 K , 若 $\text{MAX}(a, b) \leq K$, XIAO7 则认为 (a, b) 是一对奇怪的排列。

给定整数 n 和 K , 求有多少对 1 到 n 的奇怪的排列。由于答案可能很大, 你只需要输出答案对 998 244 253 取模后的结果。

输入格式

一行两个整数 n, K ($1 \leq n \leq 50, 0 \leq K \leq 2500$), 含义同题目描述。

输出格式

一行一个非负整数, 表示答案对 998 244 253 取模后的结果。

输入样例

2 3

输出样例

2

输入样例

2 3

输出样例

2

样例解释

1 到 2 的排列 $[1, 2]$ 和 $[2, 1]$ 两种, 对于 4 对排列来说:

- $\text{MAX}([1, 2], [1, 2]) = 1 + 2 = 3$;
- $\text{MAX}([1, 2], [2, 1]) = 2 + 2 = 4$;
- $\text{MAX}([2, 1], [1, 2]) = 2 + 2 = 4$;
- $\text{MAX}([2, 1], [2, 1]) = 2 + 1 = 3$;

因此满足 $\text{MAX}(a, b) \leq 3$ 的奇怪的排列有两对。

Hint

XIAO7: 发出喵喵的声音~



AUTHOR: YUKI

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<assert.h>
using namespace std;
typedef long long ll;
```

```

typedef pair<int, ll> pi;
const int N = 55;
const int inf = 0x3f3f3f3f;
const int mod = 998244253;
int getint(){
    char c; bool flag=0; int num=0;
    while((c=getchar())<'0' || c>'9') if(c=='-') flag=1;
    while(c>='0' && c<='9'){num=num*10+c-48; c=getchar();}
    return flag ? -num : num;
}
void add(int &a, int b) {
    a += b; if(a >= mod) a -= mod;
}
void sub(int &a, int b) {
    a -= b; if(a < 0) a += mod;
}
int dp[N][N][N * N];
int main(){

    int n = getint(), k = getint();
    int mid = (n + 1) >> 1, len = n >> 1;
    int Max = (n & 1) * mid + (mid + 1 + n) * len;
    dp[1][0][n] = n;
    dp[0][1][2 * n] = n * (n - 1);
    for(int i = 1; i < n; i++) {
        int val = n - i;
        for(int full = 0; full <= i; full++) {
            int half = i - full; //half = (2 * i - full * 2) / 2
            int empty = n - full - 2 * half;
            if(empty < 0) continue;
            for(int sum = 0; sum <= Max; sum++) if(dp[full][half][sum]) {
                if(full + 1 <= n && sum + val <= Max) {
                    //两个i放成一个新的full
                    add(dp[full + 1][half][sum + val],
                        1ll * dp[full][half][sum] * empty % mod);
                    //一个放half, 一个放empty, 生成一个full
                    add(dp[full + 1][half][sum + val],
                        1ll * dp[full][half][sum] * empty % mod * half * 2 % mod);
                }
                //两个i都放在half, 生成两个full, 减少一个half
                if(full + 2 <= n && half)
                    add(dp[full + 2][half - 1][sum],
                        1ll * dp[full][half][sum]
                        * (1ll * half * half % mod) % mod);
                //两个分别放在不同的empty中, 增加一个half
                if(half + 1 <= n && sum + 2 * val <= Max)
                    add(dp[full][half + 1][sum + 2 * val],
                        1ll * dp[full][half][sum] *
                        (1ll * empty * (empty - 1) % mod) % mod);
            }
        }
    }

    int ans = 0;
    for(int i = 0; i <= k; i++)

```

```
        add(ans, dp[n][0][i]);
    printf("%d\n", ans);
}
```

C5

A

A 小水獭和最长最短路

时间限制：2000ms 内存限制：65536kb

通过率：161/177 (90.96%) 正确率：161/422 (38.15%)

题目描述



小水獭发现一种奇特的路径，名为最长最短路。

具体来说，给定一个 n 个点 m 条边的无向连通图，设节点为 $1, 2, \dots, n$ ， $d(i, j)$ 表示节点 i 到节点 j 的最短路长度，记 $M = \max_{1 \leq i, j \leq n} d(i, j)$ ，请你按字典序从小到大输出所有点对 (i, j) 满足 $d(i, j) = M$ 。

(i, j) 的字典序小于 (i', j') 当且仅当 $i < i'$ 或 $(i = i') \wedge (j < j')$ 。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 300, n - 1 \leq m \leq \frac{n(n-1)}{2}$)，含义同题目描述。

接下来 m 行，每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^6$)，表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

保证给出的图没有重边和自环，且是连通的。

输出格式

对于每组数据，假设有 k 个点对满足条件，先输出一行两个正整数 M 和 k ，再输出 k 行，每行两个正整数表示对应的点对。你需要保证输出的顺序满足按字典序从小到大。

输入样例

```
2
3 2
1 2 1
1 3 1
3 3
1 3 1
1 2 1
2 3 1
```

输出样例

```
2 2
2 3
3 2
1 6
1 2
1 3
2 1
2 3
3 1
3 2
```

```
2
3 2
1 2 1
1 3 1
3 3
1 3 1
1 2 1
2 3 1
```

```
2
3 2
1 2 1
1 3 1
3 3
1 3 1
1 2 1
2 3 1
```

```
#include <bits/stdc++.h>
#define maxn 305

using namespace std;

int t, n, m;
int d[maxn][maxn];

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) d[i][j] = i == j ? 0
: 1e9;
        while(m--){
            int x, y, z;
            scanf("%d%d%d", &x, &y, &z);
            d[x][y] = d[y][x] = z;
        }
        for(int k = 1; k <= n; k++) for(int i = 1; i <= n; i++) for(int j = 1; j <=
n; j++) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        int M = 0, k = 0;
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) M = max(M, d[i][j]);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) if(d[i][j] == M)
k++;
        printf("%d %d\n", M, k);
        for(int i = 1; i <= n; i++) for(int j = 1; j <= n; j++) if(d[i][j] == M)
printf("%d %d\n", i, j);
    }
}
```

B 计算几何大难题

时间限制: 1000ms 内存限制: 65536kb

通过率: 123/176 (69.89%) 正确率: 123/788 (15.61%)

题目描述

给定平面上三个点，求这三个点围成的三角形的面积。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$)，表示数据组数。

对于每组数据，共三行，每行两个整数 ($0 \leq x_i, y_i \leq 10^9$)，表示第 i 个点的坐标。保证三点不共线。

输出格式

对于每组数据，输出一行一个浮点数，保留一位小数，表示三个点围成的三角形面积。

输入样例

```
1
0 0
0 1
1 0
```

输出样例

```
0.5
```

Hint

本题采用浮点数会有精度问题，请考虑其他做法。

```
#include <bits/stdc++.h>

typedef long long ll;

using namespace std;

inline int read(){
    int x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
    while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
    return x * f;
}

int x[3], y[3];

void solve(){
    for(int i = 0; i < 3; i++){
        x[i] = read(), y[i] = read();
    }
    for(int i = 1; i < 3; i++) y[i] -= y[0], x[i] -= x[0];
    ll ans = abs(1ll * y[2] * x[1] - 1ll * y[1] * x[2]);
    printf("%lld.%.1d\n", ans / 2, ans & 1 ? 5 : 0);
    assert(ans != 0);
}

int main(){
    int t = read();
    while(t--){
```

```
        solve();
    }
    return 0;
}
```

C

C 这是一道凸包模板题

时间限制: 1000ms 内存限制: 65536kb

通过率: 171/172 (99.42%) 正确率: 171/299 (57.19%)

题目描述

如题，这是一道凸包的模板题，在二维平面中有 n 个点，请你求出这 n 个点构成凸包的周长。

输入格式

第一行一个正整数 t ($1 \leq t \leq 50$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($3 \leq n \leq 10^3$)，表示平面中点的个数。

接下来 n 行，每行两个整数 x_i, y_i ($0 \leq x_i, y_i \leq 10^9$)，表示第 i 个点的坐标。

保证任意三点不共线。

输出格式

对于每组数据，输出一行一个浮点数，表示 n 个点构成凸包的周长。当且仅当你的答案和正确答案的绝对误差或相对误差不超过 10^{-6} 时，你的答案会被视为正确，即假设你的答案为 a ，正确答案为 b ，当且仅当 $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$ 时你的答案会被视为正确答案。

```
2
4
0 0
0 1
1 0
1 1
5
0 0
1 2
-2 -1
-1 3
0 2
```

```
4.0000000000
10.8313095581
```

```
#include<bits/stdc++.h>
using namespace std;
const int N = 2000;
int a[N];
const double eps = 1e-8;
const double Pi = acos(-1.0);
int dcmp(const double x) {
    if(fabs(x)<=eps)return 0;
    return x>0?1:-1;
}
```

```

struct Point {
    double x,y;
    Point() {}
    Point(double x,double y):x(x),y(y) {}
    Point operator + (const Point&a)const {
        return Point(x+a.x,y+a.y);
    }
    Point operator - (const Point&a)const {
        return Point(x-a.x,y-a.y);
    }
    Point operator * (const double&a)const {
        return Point(x*a,y*a);
    }
    Point operator / (const double&a)const {
        return Point(x/a,y/a);
    }
    double operator * (const Point&a)const {
        return x*a.x+y*a.y;
    }
    double operator ^ (const Point&a)const {
        return x*a.y-y*a.x;
    }
    bool operator < (const Point&a)const {
        return (x!=a.x)?x<a.x:y<a.y;
    }
    bool operator == (const Point&a)const {
        return !dcmp(x-a.x)&&!dcmp(y-a.y);
    }
} p[N],s[N];
typedef Point Vector;
double lth(const Vector a) {
    return sqrt(a*a);
}
double S_(const Point a,const Point b,const Point c) {
    return ((b-a)^(c-a))/2;
}
int build(int n,Point *p,Point *s) {
    sort(p,p+n);
    int m = 0;
    for(int i = 0; i < n; i++) {
        while(m>1&&((s[m]-s[m-1])^(p[i]-s[m-1]))<=0)m--;
        s[++m] = p[i];
    }
    int lim = m;
    for(int i = n-2; i >= 0; i--) {
        while(m>lim&&((s[m]-s[m-1])^(p[i]-s[m-1]))<=0)m--;
        s[++m] = p[i];
    }
    if(m!=1)m--;
    return m;
}
int main() {
    int t=1,n;
    long long x1,y1,x2,y2,x3,y3;
    scanf("%d",&t);

```

```

while(t--){
    scanf("%d",&n);
    for(int i=0; i<n; i++){
        scanf("%lf%lf",&p[i].x,&p[i].y);
    }
    int m=build(n,p,s);
    s[0]=s[m];
    double ans=0;
    for(int i=1; i<=m; i++){
        ans+=lth(s[i]-s[i-1]);
    }
    printf("%.10lf",ans);
}
}

```

D

D Template of Geometry

时间限制: 1000ms 内存限制: 65536kb

通过率: 6/16 (37.50%) 正确率: 6/62 (9.68%)

题目描述

Bazoka13 在整理 [We will fail] 队的计算几何模板, 但是准备考研占据了 Bazoka13 大量时间, 于是他请莫卡来帮他整理关于直线和线段位置关系部分的模板。然而莫卡对计算几何完全不了解, 于是她想请聪明的你帮她实现对直线和线段位置关系的判断。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10^5$), 表示数据组数。

对于每组数据, 共两行, 每行五个整数 $type, x_s, y_s, x_e, y_e$ ($type \in \{1, 2\}$, $-10^5 \leq x_s, y_s, x_e, y_e \leq 10^5$), 分别表示两条直线或线段; 其中后四个整数代表二维平面上两个点 $S(x_s, y_s)$ 和 $E(x_e, y_e)$ 的坐标, 保证两点坐标不同; 如果 $type = 1$, 表示该行数据为一条过点 S 和点 E 的直线, 如果 $type = 2$, 表示该行数据为一条以点 S 和点 E 为端点的线段。

输出格式

对于每组数据, 输出一行:

- 若为两条直线, 如果两条直线平行不共线, 输出 `Parallel`; 如果共线, 输出 `Coincide`; 如果相交, 输出 `Intersect`。
- 若为一条直线和一条线段, 如果非严格相交 (即在线段的端点处相交、或共线), 输出 `Not strict intersect`; 如果严格相交, 输出 `Strict intersect`; 如果不相交, 输出 `Don't intersect`。
- 若为两条线段, 如果非严格相交 (即在线段的端点处相交、或共线), 输出 `Not strict intersect`; 如果严格相交, 输出 `Strict intersect`; 如果不相交, 输出 `Don't intersect`。

输入样例

```

4
1 0 0 2 0
2 1 0 3 0
1 0 0 2 0
1 -1 -1 1 2
2 0 0 2 0
2 1 0 3 0
1 0 0 2 0
1 1 0 3 0

```

```

Not strict
Intersect
Not strict
Coincide

```

```

#include<bits/stdc++.h>
using namespace std;
#define inf 1e100
#define ll long long
#define eps 1e-8
//ÓÃÓÚ,;µãÊÝÕÝ,°ÀÐ¶İİf¬,ù¼ÝİâÄ¿¼«¶ÊÐÐ,Ä
const double pi = acos(-1.0);//Ô²ÖÜÂÊ
int sgn(ll x) {
    if(x==0)return 0;
    return x<0?-1:1;
}
//Ê¹ÓÃPointÊ±x¢Ôâ²¿·Ö°ÊÝÊÇ·µ»ØÐÃPoint¶ø·ÇÐÐ,Ä±¼ÉíÖµ
struct Point {
    ll x,y;
    /*¹¹Ôì°ÊÝ*/
    Point() {}
    Point(ll xx,ll yy) {
        x=xx;
        y=yy;
    }
    /*ÖÖÖÖ»Ð@µãµÄ»ù´;ÔÊÊã·û*/
    bool operator == (Point b)const {
        return sgn(x-b.x) == 0 && sgn(y-b.y) == 0;
    }
    bool operator < (Point b)const {
        return sgn(x-b.x)== 0?sgn(y-b.y)<0:x<b.x;
    }
    Point operator -(const Point &b)const {
        return Point(x-b.x,y-b.y);
    }
    Point operator +(const Point &b)const {
        return Point(x+b.x,y+b.y);
    }
    //²æ»ý
    ll operator ^(const Point &b)const {
        return x*b.y - y*b.x;
    }
    //µã»ý
    ll operator *(const Point &b)const {
        return x*b.x + y*b.y;
    }

    void input() {
        scanf("%lld%lld",&x,&y);
    }
};
struct Line {
    //Ã½µãÊ·¶Ö±İß
    Point s,e;
    Line() {}
    Line(Point ss,Point ee) {
        s=ss;
        e=ee;
    }

```

```

}
void input() {
    s.input();
    e.input();
}
//µãðúîßŋîéî
bool checkPS(Point p) {
    return sgn((p-s)^(e-s)) == 0 && sgn((p-s)*(p-e)) <= 0;
}
//ö±îßÆ½ðð
bool parallel(Line v) {
    return sgn((e-s)^(v.e-v.s)) == 0;
}
//µã°íö±îß'øîµ
//1  ôú×ó²à
//2  ôúôð²à
//3  ôúö±îßéî
int relation(Point p) {
    int c = sgn((p-s)^(e-s));
    if(c < 0)return 1;
    else if(c > 0)return 2;
    else return 3;
}
//îßŋîîà½»
//2  'æ·ŋîîà½»
//1  ·ç¹æ·ŋîîà½»
//0  ²»îà½»
int checkSS(Line v) {
    int d1 = sgn((e-s)^(v.s-s));
    int d2 = sgn((e-s)^(v.e-s));
    int d3 = sgn((v.e-v.s)^(s-v.s));
    int d4 = sgn((v.e-v.s)^(e-v.s));
    if( (d1^d2)==-2 && (d3^d4)==-2 )return 2;
    return (d1==0 && sgn((v.s-s)*(v.s-e))<=0) ||
           (d2==0 && sgn((v.e-s)*(v.e-e))<=0) ||
           (d3==0 && sgn((s-v.s)*(s-v.e))<=0) ||
           (d4==0 && sgn((e-v.s)*(e-v.e))<=0);
}
//ö±îß°íîßŋîîà½»
//2  'æ·ŋîîà½»
//1  ·ç¹æ·ŋîîà½»
//0  ²»îà½»
int checkLS(Line v) {
    int d1 = sgn((e-s)^(v.s-s));
    int d2 = sgn((e-s)^(v.e-s));
    if((d1^d2)==-2) return 2;
    return (d1==0||d2==0);
}
//Á½ö±îß'øîµ
//0  Æ½ðð
//1  öð°î
//2  îà½»
int checkLL(Line v) {
    if((*this).parallel(v))
        return v.relation(s)==3;
}

```

```

        return 2;
    }
};

int main() {
    int t;
    scanf("%d",&t);
    while(t--) {
        int type1,type2,xs,ys,xs,ys,xs,ys,xs,ys;
        Line l1, l2;
        scanf("%d%d%d%d",&type1,&xs,&ys,&xe,&ye);
        l1 = Line(Point(xs,ys),Point(xe,ye));
        scanf("%d%d%d%d",&type2,&xs,&ys,&xe,&ye);
        l2 = Line(Point(xs,ys),Point(xe,ye));
        if(type1 == 1 && type2 == 1) {
            if(l1.checkLL(l2) == 0) {
                puts("Parallel");
            } else if(l1.checkLL(l2) == 1) {
                puts("Coincide");
            } else {
                puts("Intersect");
            }
        }
        else if(type1 == 1) {
            if(l1.checkLS(l2) == 2) {
                puts("Strict");
            } else if(l1.checkLS(l2) == 1) {
                puts("Not strict");
            } else {
                puts("Don't intersect");
            }
        }
        else if(type2 == 1) {
            if(l2.checkLS(l1) == 2) {
                puts("Strict");
            } else if(l2.checkLS(l1) == 1) {
                puts("Not strict");
            } else {
                puts("Don't intersect");
            }
        }
        else {
            if(l1.checkSS(l2) == 2) {
                puts("Strict");
            } else if(l1.checkSS(l2) == 1) {
                puts("Not strict");
            } else {
                puts("Don't intersect");
            }
        }
    }
}

```

题目描述

XIAO7 不想努力了，于是她决定抓几只草兽来帮忙干活。

二维平面上有 n 只草兽，坐标分别为 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ 。

XIAO7 有一颗智慧宝珠，可以对一个圆形范围内的草兽进行计数。她会进行 q 次询问，每次询问会给定三个参数 X, Y, R ，你需要求解有多少草兽其坐标 (x_i, y_i) 满足 $(x_i - X)^2 + (y_i - Y)^2 \leq R^2$ 。

输入格式

第一行两个正整数 n, q ($1 \leq n, q \leq 5 \times 10^4$)，表示草兽的数量和询问的次数。

接下来 n 行，每行两个整数 x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$)，表示草兽的坐标。可能有多个草兽坐标相同。

接下来 q 行，每行三个整数 X_i, Y_i, R_i ($1 \leq X_i, Y_i \leq 10^9, 1 \leq R_i \leq 10$)，表示第 i 次询问的参数。

输出格式

输出 q 行，第 i 行一个非负整数，表示第 i 次询问的答案。

```
5 3
0 0
1 0
1 1
1 2
2 2
1 1 1
3 3 2
3 2 2
```

```
3
1
2
```

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<cstdlib>
#include<algorithm>
#include<vector>
#include<cstring>
#include<set>
#include<ctime>
#include<map>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
const int N = 100005;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
```

```

}
map<pi, int> point;
ll sqr(int x) {
    return 1ll * x * x;
}
ll dis(int x1, int y1, int x2, int y2) {
    return sqr(x1 - x2) + sqr(y1 - y2);
}
int main(){

    int n = getint(), q = getint();

    for(int i = 1; i <= n; i ++) {
        int x = getint(), y = getint();

        if(point.count(make_pair(x, y)))
            point[make_pair(x, y)] ++;
        else
            point[make_pair(x, y)] = 1;
    }

    while(q --) {
        int x = getint(), y = getint(), r = getint(), ans = 0;

        for(int i = x - r; i <= x + r; i ++)
            for(int j = y - r; j <= y + r; j ++)
                if(dis(i, j, x, y) <= sqr(r) && point.count(make_pair(i, j))) {

                    ans += point[make_pair(i, j)];
                }
        printf("%d\n", ans);
    }
}

```

F

F 上班 III

时间限制: 1000ms 内存限制: 65536kb

通过率: 55/81 (67.98%) 正确率: 55/306 (17.97%)

10

4957541

4957260

4957192

题目描述

观赛完运动会之后, Zhoues 需要立马回去钢厂上班, 由于北航到钢厂的路径有很多条, 他希望选择一条最短的!

但是由于最近海淀区疫情比较严重, 钢厂要求回去上班的员工必须做一次核酸检测, 因此 Zhoues 必须要在 k 个核酸检测点 $s_1, s_2, \dots, s_k$ 中选取一个核酸检测点做完核酸之后才可以上班。

形式化地说, 给定一个 n 个点 m 条边的无向图和 k 个指定节点, 要求在至少经过一个指定节点的情况下, 求点 1 到点 n 的最短路长度。

输入格式

第一行三个正整数 n, m, k ($1 \leq n \leq 10^5, 1 \leq m \leq 5 \times 10^5, 1 \leq k \leq n$), 含义同题目描述。

接下来 m 行, 每行三个正整数 u_i, v_i, w_i ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^3$), 表示第 i 条边 (u_i, v_i) 的权值为 w_i 。

接下来 k 行, 每行一个正整数 s_i ($1 \leq s_i \leq n$), 表示第 i 个指定节点。保证这 k 个节点不同。

图中有可能存在重边和自环, 保证至少存在一条从点 1 经过指定节点并到点 n 的路径。

输出格式

一行一个非负整数, 表示在至少经过一个指定节点的情况下, 点 1 到点 n 的最短路长度。

```
7 6 1
1 2 1
1 3 2
2 4 2
2 5 5
3 6 3
3 7 4
4
```

12

样例解释

在必须经过点 44 的情况下, 最短路径为 $1 \rightarrow 2 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 7 \rightarrow 2 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 7$ 。

Hint

单源最短路 $\rightarrow$ 「双源」最短路。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

ll read() {
    ll x=0,w=1; char c=getchar();
    while (c<'0' || c>'9') { if (c=='-') w=-1; c=getchar(); }
    while (c>='0' && c<='9') x=x*10+c-'0', c=getchar();
    return x*w;
}

const ll N=1000000+10, M=5000000+10;

ll n,m,k,t,ans = LLONG_MAX;
struct edge { ll v,w,nxt; } e[M];
ll head[N];
void addEdge(ll u,ll v,ll w) {
```

```

static ll cnt=0;
e[++cnt]=(edge){v,w,head[u]},head[u]=cnt;
}

priority_queue<pair<ll,ll>,vector<pair<ll,ll>>,greater<pair<ll,ll>>> Q;
ll dis[N];
ll dis2[N];
void dijkstra(ll s) {
    memset(dis,0x3f,sizeof(dis));
    dis[s]=0,Q.push(make_pair(0,s));
    while (!Q.empty()) {
        ll u=Q.top().second,d=Q.top().first; Q.pop();
        if (d!=dis[u]) continue;
        for (ll i=head[u];i;i=e[i].nxt) {
            ll v=e[i].v,w=e[i].w;
            if (dis[u]+w<dis[v]) dis[v]=dis[u]+w,Q.push(make_pair(dis[v],v));
        }
    }
}
void dijkstra2(ll s) {
    memset(dis2,0x3f,sizeof(dis2));
    dis2[s]=0,Q.push(make_pair(0,s));
    while (!Q.empty()) {
        ll u=Q.top().second,d=Q.top().first; Q.pop();
        if (d!=dis2[u]) continue;
        for (ll i=head[u];i;i=e[i].nxt) {
            ll v=e[i].v,w=e[i].w;
            if (dis2[u]+w<dis2[v])
dis2[v]=dis2[u]+w,Q.push(make_pair(dis2[v],v));
        }
    }
}
int main() {
    n=read(),m=read(),k=read();
    for (ll i=1;i<=m;++i) {
        ll u=read(),v=read(),w=read();
        addEdge(u,v,w);
        addEdge(v,u,w);
    }
    dijkstra(1);
    dijkstra2(n);
    for (ll i=0;i<k;++i) {
        int t = read();
        ans = min(ans,dis[t] + dis2[t]);
    }
    printf("%lld",ans);
    return 0;
}

```

G 小水獭和贝叶斯网络

时间限制：2000ms 内存限制：65536kb

通过率：4/6 (66.67%) 正确率：4/13 (30.77%)

题目描述



小水獭接触到了贝叶斯网络，它现在有一个有向图，希望求出一个边数最多的无环生成子图作为贝叶斯网络。可以证明这个问题是 **NP** 的，因此小水獭希望你得到一个近似比为 $\frac{1}{2}$ 的算法。

具体来说，给定一个 n 个点 m 条边的有向图，请你保留全部的节点和至少 $\left\lceil \frac{m}{2} \right\rceil$ 条边，使得图中无环。

输入格式

第一行一个正整数 t ($1 \leq t \leq 10$)，表示数据组数。

对于每组数据，第一行两个正整数 n, m ($1 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$)，含义如题目所示。

接下来 m 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示第 i 条边为 $u_i \rightarrow v_i$ 。

保证给出的图没有重边和自环。

输出格式

对于每组数据，先输出一行一个正整数 k ($\left\lceil \frac{m}{2} \right\rceil \leq k \leq m$)，表示你所保留的边数。

接下来 k 行，每行两个正整数 u_i, v_i ($1 \leq u_i, v_i \leq n$)，表示保留的第 i 条边为 $u_i \rightarrow v_i$ 。

如果有多种可能的答案，你可以输出任意一种。

输入样例

```
2
4 4
1 2
1 3
1 4
2 4
4 4
1 2
2 3
3 4
4 1
```

输出样例

```
4
1 2
1 3
1 4
2 4
3
1 2
2 3
4 1
```

```
#include <bits/stdc++.h>
#define maxn 100086

using namespace std;

int t, n, m;

int main(){
    scanf("%d", &t);
    while(t--){
        scanf("%d%d", &n, &m);
        vector<pair<int, int> > v[2];
        while(m--){
```

```
        int x, y;
        scanf("%d%d", &x, &y);
        v[x < y].push_back({x, y});
    }
    if(v[0].size() < v[1].size()) swap(v[0], v[1]);
    printf("%d\n", v[0].size());
    for(auto it : v[0]) printf("%d %d\n", it.first, it.second);
}
}
```

H

H 平面最近点对 II

时间限制：2000ms 内存限制：204800kb

通过率：124/154 (80.52%) 正确率：124/427 (29.04%)

题目描述

Zhoues 发现二维平面上有 n 个点，坐标分别为 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ ，定义两点 (x_i, y_i) 和 (x_j, y_j) 的欧几里得距离为 $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ 。Zhoues 要求同学们求出任意两点间最小欧几里得距离，即：

$$\min_{1 \leq i < j \leq n} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

输入格式

第一行一个正整数 n ($2 \leq n \leq 5 \times 10^5$)，表示点的数量。

接下来 n 行，每行两个至多六位小数的浮点数 x_i, y_i ($0 \leq |x_i|, |y_i| \leq 10^6$)，表示第 i 个点的坐标。

输出格式

输出一行一个浮点数，保留四位小数，表示任意两点间最小欧几里得距离。

输入样例

```
3
0.00 0.00
2.00 2.00
1.00 3.00
```

输出格式

输出一行一个浮点数，保留四位小数，表示任意两点间最小欧几里得距离。

输入样例

```
3
0.00 0.00
2.00 2.00
1.00 3.00
```

输出样例

```
1.4142
```

Hint

经典分治问题，本题的所有伪代码具体可以参考算法导论 P613、课件 PPT 和仓库中 C2-A 的题解。

```
#include<iostream>
#include<cstdio>
#include<cstring>
#include<cmath>
#include<algorithm>
```

```

using namespace std;
struct Point{double x,y;}a[2000000],tmp[2000000];
int n,b[2000000];
const double inf=0x7fffffff;
double sqr(double x){return x*x;}
double dis(int l,int r){return sqr(a[l].x-a[r].x)+sqr(a[l].y-a[r].y);}
int cmp(const Point &a,const Point &b){return a.x==b.x?a.y<b.y:a.x<b.x;}
void merge(int l,int r)
{
    int mid=(l+r)>>1,i=l,j=mid+1;
    for(int k=l;k<=r;k++)
    {
        if(i<=mid&&(j>r||a[i].y<a[j].y))tmp[k]=a[i++];
        else tmp[k]=a[j++];
    }
    for(int i=l;i<=r;i++)a[i]=tmp[i];
}
double solve(int l,int r)
{
    if(l>=r)return inf;
    if(l+1==r){if(a[l].y>a[r].y)swap(a[l],a[r]);return dis(l,r);}
    int mid=(l+r)>>1,t=a[mid].x,cnt=0;
    double d=min(solve(l,mid),solve(mid+1,r));
    merge(l,r);
    for(int i=l;i<=r;i++)
        if(sqr(a[i].x-t)<d)
            b[++cnt]=i;
    for(int i=1;i<=cnt;i++)
        for(int j=i+1;j<=cnt&&sqrt(a[b[j]].y-a[b[i]].y)<d;j++)
            d=min(d,dis(b[j],b[i]));
    return d;
}
int main()
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++)scanf("%lf%lf",&a[i].x,&a[i].y);
    sort(a+1,a+n+1,cmp);
    printf("%.4lf",sqrt(solve(1,n)));
}

```

I XIAO7 和猫瘟公司

时间限制：3000ms 内存限制：131072kb

通过率：4/7 (57.14%) 正确率：4/13 (30.77%)

题目描述

XIAO7 的猫娘乐园里发生了猫瘟病毒泄露，她需要尽快逃出去！

猫娘乐园里一共有 n 栋猫舍，猫舍之间有 m 条单向的道路，每条道路都有一个长度和年龄。雪风所在的 1 号猫舍发生了病毒泄露，位于 n 号猫舍的 XIAO7 在得知情况后决定抛弃猫娘乐园逃跑，总共需要 T 秒才能彻底逃离。

为了帮助 XIAO7 成功逃离，你需要拆除猫娘乐园的部分道路，使得从 1 号猫舍到 n 号猫舍的最短路长度至少为 T （或使得 1 号猫舍无法到达 n 号猫舍）。拆除道路对猫娘乐园的破坏程度定义为被拆除的道路中最古老的一条路的年龄。XIAO7 希望在能成功逃离的基础上对猫娘乐园的破坏程度尽可能小，请你帮帮她！

输入格式

第一行三个整数 n, m, T ($1 \leq n \leq 5 \times 10^4$, $1 \leq m \leq 2 \times 10^5$, $0 \leq T \leq 10^9$)，含义同题目描述。

接下来 m 行，每行四个正整数 u_i, v_i, l_i, a_i ($1 \leq u_i, v_i \leq n$, $1 \leq l_i, a_i \leq 10^9$)，表示第 i 条边 $u_i \rightarrow v_i$ 的长度和年龄分别为 l_i 和 a_i 。

图中有可能存在重边和自环，保证至少存在一条从 1 号猫舍到 n 号猫舍的路径。

输出格式

输出一行：

- 若需要拆除道路则输出一个正整数，表示最小破坏程度。
- 否则，输出 `0 v 0` 和从 1 到 n 的最短路长度（显然，它应该 $\geq T$ ）。

输入样例 1

```
5 6 12
1 2 5 10
2 3 6 20
1 4 4 15
4 3 8 25
3 5 2 18
4 5 7 12
```

输出样例 1

12

输入样例 2

```
3 2 10
1 2 8 30
2 3 6 45
```

输出样例 2

0 v 0 14

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<ll, int> pi;
const int N = 100005;
const int M = 2000005;
const ll inf = 1ll << 60;
int getint(){
    char c;bool flag=0;int num=0;
    while((c=getchar())<'0' || c>'9')if(c=='-')flag=1;
    while(c>='0'&&c<='9'){num=num*10+c-48;c=getchar();}
    return flag ? -num : num;
}
int n, m, T, cnt, fir[N], tar[M], nxt[M], w[M], age[M], A[M];
ll dis[N]; bool vis[N];
priority_queue<pi, vector<pi>, greater<pi> > q;
```

```

void link(int a, int b, int c, int A) {
    tar[++ cnt] = b, w[cnt] = c, age[cnt] = A;
    nxt[cnt] = fir[a], fir[a] = cnt;
}

bool dij(int x, int ed, int mx) {
    for(int i = 1; i <= n; i ++)
        dis[i] = inf, vis[i] = 0;
    q.push(make_pair(dis[x] = 0, x));
    while(! q.empty()) {
        x = q.top().second, q.pop();
        if(vis[x]) continue ;

        vis[x] = 1;
        for(int i = fir[x]; i; i = nxt[i])
            if(age[i] > mx && dis[tar[i]] > dis[x] + w[i]) {
                dis[tar[i]] = dis[x] + w[i];
                q.push(make_pair(dis[tar[i]], tar[i]));
            }
    }
    return dis[ed] >= T;
}

int main(){

    n = getint(), m = getint(), T = getint();
    for(int i = 1; i <= m; i ++) {
        int x = getint(), y = getint();
        int len = getint(), age = getint();
        link(x, y, len, A[i] = age);
    }
    sort(A + 1, A + m + 1);
    if(dij(1, n, -1)) {
        printf("o v o %lld\n", dis[n]);
        return 0;
    }

    int l = 1, r = m;
    while(l < r) {
        int mid = (l + r) >> 1;
        if(dij(1, n, A[mid]))
            r = mid;
        else l = mid + 1;
    }
    cout << A[l] << endl;
}

```

题目描述

莫卡学会了一个新的魔法——姆修派斯光线!

莫卡来到了练习场练习这个魔法。练习场可以视为一个无穷大的二维平面，并且上面分布着 n 个练习人偶。姆修派斯光线比较奇特，当有两个不同的练习人偶和莫卡在一条直线上时，莫卡可以释放该魔法并摧毁这两个练习人偶。

莫卡在练习场中标出了 q 个位置，每次练习莫卡会在当前位置不断地释放魔法并摧毁练习人偶，直到释放条件无法满足为止。之后练习人偶会在原来的位置恢复，莫卡会传送到下一个位置继续练习。

莫卡想知道，对于每个位置，她最多能够释放几次魔法？

输入格式

第一行一个整数 t ($1 \leq t \leq 20$)，表示数据组数。

对于每组数据，第一行一个正整数 n ($1 \leq n \leq 10^3$)，表示练习人偶数量。

接下来 n 行，每行两个整数 x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$)，表示第 i 个练习人偶的位置。

接下来一行一个正整数 q ($1 \leq q \leq 100$)，表示莫卡练习位置的数量。

接下来 q 行，每行两个整数 X_i, Y_i ($-10^9 \leq X_i, Y_i \leq 10^9$)，代表第 i 个练习位置的坐标。

输出格式

对于每组数据，输出 q 行，第 i 行一个非负整数，表示第 i 个练习位置最多能够释放几次魔法。

输入样例

```
1
4
2 2
-2 -2
2 -2
-2 2
2
0 0
2 0
```

输出样例

```
2
1
```

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long
int sgn(ll x) {
    if(x==0)return 0;
    return x<0?-1:1;
}
struct Point {
    ll x,y;
    Point() {}
    Point(ll xx,ll yy) {
        x=xx;
        y=yy;
    }
    void input() {
        scanf("%lld %lld",&x,&y);
    }
    bool operator ==(const Point &p)const {
        return sgn(x-p.x)==0&&sgn(y-p.y)==0;
    }
    bool operator !=(const Point &p)const {
        return sgn(x-p.x)!=0||sgn(y-p.y)!=0;
    }
};
```

```

    }
    bool operator < (const Point &p)const {
        return sgn(x-p.x)<0||(sgn(x-p.x)==0&&sgn(y-p.y)<0);
    }
    Point operator + (const Point &p)const {
        return {x+p.x,y+p.y};
    }
    Point operator - (const Point &p)const {
        return {x-p.x,y-p.y};
    }
    Point operator * (ll k)const {
        return {x*k,y*k};
    }
    Point operator / (const ll &k)const {
        return {x/k,y/k};
    }
    ll dot(Point p) const {
        return p.x*x+p.y*y;
    }
    ll cross(Point p) const {
        return x * p.y - y * p.x;
    }
};

ll cross(Point p1,Point p2,Point p3) {
    return (p2-p1).cross(p3-p1);
}

ll dot(Point p1,Point p2,Point p3) {
    return (p2-p1).dot(p3-p1);
}

struct Frac {
    ll a,b;
    Frac() {}
    Frac(ll aa,ll bb) {
        if(aa!=0&&bb!=0) {
            ll g=__gcd(abs(aa),abs(bb));
            a=aa/g;
            b=bb/g;
        }
        if(aa==0&&bb!=0) {
            a=0;
            b=bb/abs(bb);
        }
        if(aa!=0&&bb==0) {
            b=0;
            a=aa/abs(aa);
        }
    }
};

map<pair<ll,ll>,int> mp;
vector<Point> ps;
int main() {
    int t;
    scanf("%d",&t);
    while(t--) {
        int n;

```

```

    ps.clear();
    scanf("%d",&n);
    for(int i=1; i<=n; i++) {
        Point tmp;
        tmp.input();
        ps.push_back(tmp);
    }
    int q;
    scanf("%d",&q);
    while(q--) {
        Point p;
        p.input();
        mp.clear();
        for(auto i:ps) {
            Frac f=Frac(i.x-p.x,i.y-p.y);
            mp[ {f.a,f.b} ]++;
        }
        ll ans=0;
        for(auto i:ps) {
            Frac f=Frac(i.x-p.x,i.y-p.y);
            int sz=mp[ {-f.a,-f.b} ]+mp[ {f.a,f.b} ];
            if(sz>=2)ans=ans+sz/2;
            mp[ {-f.a,-f.b} ]=0;
            mp[ {f.a,f.b} ]=0;
        }
        printf("%lld\n",ans);
    }
}

```